Audio File Manipulation Project

1. Project Overview

- This project is focused on building a simple audio file manipulation system that allows loading, playing, merging, and controlling audio files.
- The codebase is written in **C++** and leverages classes for abstraction and modular design.

2. Main Components

a. AudioFile Class

- **Purpose:** Represents an audio file and provides basic functionality for audio manipulation.
- **Key Features:**
    - Load an audio file from a file path.
    - Play and stop audio.
    - Set and get volume.
    - Retrieve audio duration.
- **Code Implementation:**

b. Sampler Class

- **Purpose:** Manages multiple audio files and allows for combining them into a single output.
- **Key Features:**
    - Add multiple audio files.
    - Merge audio files into one output file.
    - Play and stop all loaded files.
    - Set global volume.
- **Code Implementation:**

c. AudioEngine Class

- **Purpose:** Acts as the core engine to initialize and manage playback operations.
- **Key Features:**
  - Initialize the audio engine.
  - Start and stop playback using the Sampler class.
- **Code Implementation:**

3. How the System Works

- **Initialization:**
  - The AudioEngine class is initialized to prepare the system for audio playback.
  - Example: audioEngine.initialize();
- **Loading Audio Files:**
  - Audio files are created using the AudioFile class, and loaded from specific paths.
  - Example:

- **Adding to Sampler:**
  - The loaded audio files are added to the Sampler for playback and manipulation.
  - Example:

- **Merging and Playback:**
  - Once files are added to the Sampler, they can be merged into one file or played simultaneously.
  - Example:

4. Practical Applications

- Audio production tools can benefit from this system for merging multiple audio tracks.
- Could be used in music software, games, or apps that require sound manipulation.
- Extensibility to add more features like audio effects, pitch control, and format conversions.

5. Future Improvements

- **Advanced Manipulation:** Add support for effects such as reverb, delay, and equalization.
- **File Format Support:** Enhance to support different audio file formats like MP3, FLAC.
- **Real-Time Playback Control:** Implement more granular control over real-time playback operations.

6. Conclusion

- The project provides a foundational structure for audio file manipulation using object-oriented principles.
- It's flexible and can be expanded to support more advanced features in future development.

GitHub

PLAY
PLAY ALL
MERGER
MENGE
PLAYAY
AUDIO SAMPLER
PLAY ALL
VOLUME
PLAY STOP
LAY ALL
STOP ALL
PLAYPAD
PLAY
MERGE
PLAY ALL PLAY ALL
PLAY STOP ALL
PLAY STOP ALL
STOP ALL
MERGE
MERGE
MERGE